S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 04

Aim: Make use of A* Algorithm to find out goal state (Shortest path cost) in the given Graph. Make use of Breadth First Search algorithm to find the goal state in the above graph and compare the performance of both algorithms in terms of time taken.

Name of Student :

USN No. :

Semester/Year: VI/III

Academic Session: 2025-2026

Date of Performance:

Date of Submission:

AIM: Make use of A* Algorithm to find out goal state (Shortest path cost) in the given Graph. Make use of Breadth First Search algorithm to find the goal state in the above graph and compare the performance of both algorithms in terms of time taken.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

· To find the path from start node to goal node using A* Searching Algorithm technique.

· To analyze the performance of A* searching Algorithm.

THEORY:

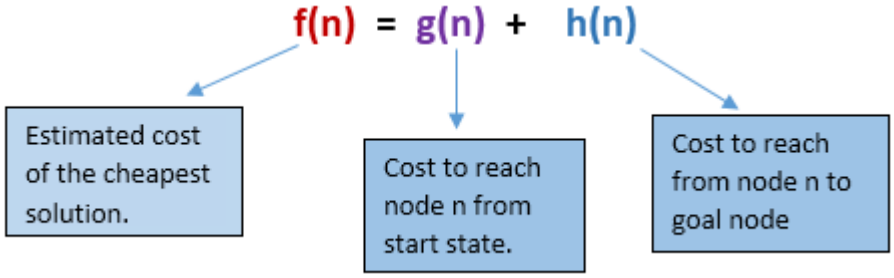
A* Algorithm in Python or in general is basically an artificial intelligence problem used for the path finding (from point A to point B) and the Graph traversals. This algorithm is flexible and can be used in a wide range of contexts. The A* search algorithm uses the heuristic path cost, the starting point's cost, and the ending point.

Why A* Algorithm?

This Algorithm is the advanced form of the BFS algorithm (Breadth-first search), which searches for the shorter path first than, the longer paths. It is a complete as well as an optimal solution for solving path and grid problems.

Optimal – find the least cost from the starting point to the ending point. Complete – It means that it will find all the available paths from start to end.

Basic concepts of A*



G(n)=h(n)+f(n)

Where,

g (n) : The actual cost path from the start node to the current node.

h (n) : The actual cost path from the current node to goal node.

f (n) : The actual cost path from the start node to the goal node.

For the implementation of A* algorithm we have to use two arrays namely OPEN and CLOSE.

OPEN:
An array that contains the nodes that have been generated but have not been yet examined till yet.

CLOSE:
An array which contains the nodes which are examined.

Advantages of A* Algorithm in Python

· It is fully complete and optimal.

· This is the best one of all the other techniques. We use to solve all the complex problems through this algorithm.

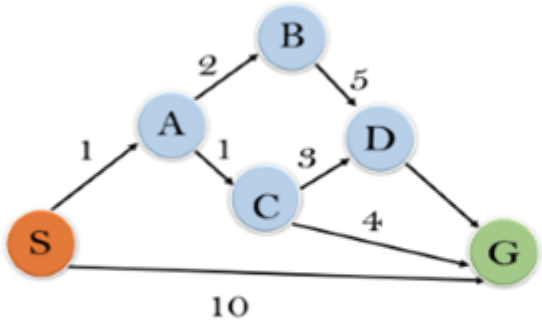
· The algorithm is optimally efficient, i.e., there is no other optimal algorithm that is guaranteed to expand fewer nodes than A*.

Disadvantages of A* Algorithm in Python

· This algorithm is complete if the branching factor is finite of the algorithm and every action has a fixed cost.

· The speed execution of A* search is highly dependent on the accuracy of the heuristic algorithm that is used to compute h (n) and is a bit slower than other algorithms. · It is having complex problems.

Example:



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

PROGRAM CODE/PSEUDO:

· START
· Let openlist equal empty list of nodes
· Let closedlist equal empty list of nodes
· Put startnode on the openlist (leave it's f at zero)
· While openlist is not empty
· Let currentnode equal the node with the least f value
· Remove currentnode from the openlist
· Add currentnode to the closedlist
· If currentnode is the goal
· You've found the exit!
· Let children of the currentnode equal the adjacent nodes
· For each child in the children
· If child is in the closedlist
· Continue to beginning of for loop
· Child.g = currentnode.g + distance b/w child and current
· Child.h = distance from child to end
· Child.f = child.g + child.h
· If child.position is in the openlist's nodes positions if child.g
· Is higher than the openlist node's g
· Continue to beginning of for loop
· Add the child to the openlist
· END

ALGORITHM:

1: Firstly, Place the starting node into OPEN and find its f (n) value.

2: Then remove the node from OPEN, having the smallest f (n) value. If it is a goal node, then stop and return to success.

3: Else remove the node from OPEN, and find all its successors.

4: Find the f (n) value of all the successors, place them into OPEN, and place the removed node into CLOSE.

5: Goto Step-2.

6: Exit.

CODE :

```
import heapq
graph = {}
heuristic = {}
n = int(input("Enter number of nodes: "))
nodes = []
for i in range(n):
    node = input(f"Enter node {i+1}: ")
    nodes.append(node)
    graph[node] = []

e = int(input("Enter number of edges: "))
print("Enter edges in format: source destination cost")
for i in range(e):
    src, dest, cost = input(f"Edge {i+1}: ").split()
    graph[src].append((dest, int(cost)))

print("\nEnter heuristic values:")
for node in nodes:
    heuristic[node] = int(input(f"Heuristic of {node}: "))
```

```
start = input("\nEnter start node: ")
goal = input("Enter goal node: ")
def a_star(start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))
    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0
    parent = {}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
            return path[::-1], g_cost[goal]

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost
            if new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                f_cost = new_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                parent[neighbor] = current

    return None, float('inf')

path, cost = a_star(start, goal)
```

```
print("\nShortest Path using A*:", path)
print("Total Cost:", cost)
```

INPUT & OUTPUT:

```
Enter number of nodes: 6
Enter node 1: S
Enter node 2: A
Enter node 3: B
Enter node 4: C
Enter node 5: D
Enter node 6: G
Enter number of edges: 8
Enter edges in format: source destination cost
Edge 1: S A 1
Edge 2: S G 10
Edge 3: A B 2
Edge 4: A C 1
Edge 5: B D 5
Edge 6: C D 3
Edge 7: C G 4
Edge 8: D G 2

Enter heuristic values:
Heuristic of S: 5
Heuristic of A: 3
Heuristic of B: 4
Heuristic of C: 2
Heuristic of D: 6
Heuristic of G: 0

Enter start node: S
Enter goal node: G

Shortest Path using A*: ['S', 'A', 'C', 'G']
Total Cost: 6
```

CONCLUSION:

_____

_____

_____

_____

_____

_____

DISCUSSION QUESTIONS:

1. What are applications of A * algorithm?

_____

_____

_____

_____

_____

_____

2. Why is A * better than best-first search?

3. Why is A * called A *?

4. What is A* search used for?

5. Why is A * search algorithm preferred?

REFERENCES:

- https://www.pythonpool.com/a-star-algorithm-python/
- https://www.askpython.com/python/examples/a-star-algorithm
- https://saturncloud.io/blog/implementing-the-a-algorithm-in-python-a-stepbystep-guide/
- https://www.simplilearn.com/tutorials/artificial-intelligence-tutorial/a-star-algorithm
- https://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html